25IT103
Kavya Ukani

PRACTICAL – 2

2.1) Smart thermostat (Thermostat.java, main in this file) (a) Create class Thermostat with private String location, private int temperature, private static final int MIN=16 and MAX=30, and private static int activeCount=0. (b) Constructor(location, startTemp): store location; set temperature to startTemp if it is within MIN..MAX else 22; increment activeCount. (c) Constructor(location): chain with this(location, 22). (d) raise(): if temperature < MAX add 1, else print “Already at maximum (30)”. (e) lower(): if temperature > MIN subtract 1, else print “Already at minimum (16)”. (f) Add getTemperature() (no setter) and static getActiveCount(). (g) In main: create two thermostats, call raise() 10 times then lower() 20 times in loops (printing the temperature each time), then print getActiveCount(). Expected output: temperature rises to 30 then “Already at maximum”, falls to 16 then “Already at minimum”, count 2.

```
public class Thermostat {

    private String location;
    private int temperature;
    private static final int MIN = 16;
    private static final int MAX = 30;
    private static int activeCount = 0;

    public Thermostat(String location, int startTemp) {
        this.location = location;

        if (startTemp >= MIN && startTemp <= MAX) {
            temperature = startTemp;
```

```
        } else {
            temperature = 22;
        }

        activeCount++;
    }

    public Thermostat(String location) {
        this(location, 22);
    }

    public void raise() {
        if (temperature < MAX) {
            temperature++;
        } else {
            System.out.println("Already at maximum (30)");
        }
    }

    public void lower() {
        if (temperature > MIN) {
            temperature--;
        } else {
            System.out.println("Already at minimum (16)");
        }
    }
```

```
public int getTemperature() {
    return temperature;
}

public static int getActiveCount() {
    return activeCount;
}

public static void main(String[] args) {

    Thermostat t1 = new Thermostat("Living Room");
    Thermostat t2 = new Thermostat("Bedroom", 25);

    System.out.println("Initial Temperature: " + t1.getTemperature());

    for (int i = 0; i < 10; i++) {
        t1.raise();
        System.out.println("Temperature: " + t1.getTemperature());
    }

    for (int i = 0; i < 20; i++) {
        t1.lower();
        System.out.println("Temperature: " + t1.getTemperature());
    }
```

```
        System.out.println("Active Thermostats: " +
Thermostat.getActiveCount());
    }
}
```

2.2) Cinema show (CinemaShow.java, main in this file) (a) Create class CinemaShow with private String title, private int seatsAvailable, private final int capacity, and private static int totalBooked=0. (b) Constructor(title, capacity): set both and seatsAvailable = capacity. Constructor(title): chain with this(title, 100). (c) book(int n): if n ≤ seatsAvailable, reduce seatsAvailable by n, add n to totalBooked, return true; else return false unchanged. (d) cancel(int n): increase seatsAvailable by n but never above capacity. (e) Add getSeatsAvailable() and static getTotalBooked(). (f) In main: run a sequence of book and cancel calls, printing the result and seats after each, then print getTotalBooked(). Expected output: booking more than available prints false and leaves seats unchanged; totalBooked equals the successful bookings.

```
public class CinemaShow {

    private String title;
    private int seatsAvailable;
    private final int capacity;
    private static int totalBooked = 0;
```

```
    public CinemaShow(String title, int capacity) {
        this.title = title;
        this.capacity = capacity;
        this.seatsAvailable = capacity;
    }

    public CinemaShow(String title) {
        this(title, 100);
    }

    public boolean book(int n) {
        if (n <= seatsAvailable) {
            seatsAvailable -= n;
            totalBooked += n;
            return true;
        }
        return false;
    }

    public void cancel(int n) {
        seatsAvailable += n;
        if (seatsAvailable > capacity) {
            seatsAvailable = capacity;
        }
    }
```

```
    public int getSeatsAvailable() {
        return seatsAvailable;
    }

    public static int getTotalBooked() {
        return totalBooked;
    }

    public static void main(String[] args) {

        CinemaShow show = new CinemaShow("Avengers", 50);

        System.out.println(show.book(20));
        System.out.println("Seats Available: " + show.getSeatsAvailable());

        System.out.println(show.book(25));
        System.out.println("Seats Available: " + show.getSeatsAvailable());

        System.out.println(show.book(10));
        System.out.println("Seats Available: " + show.getSeatsAvailable());

        show.cancel(15);
        System.out.println("Seats Available: " + show.getSeatsAvailable());

        System.out.println(show.book(10));
        System.out.println("Seats Available: " + show.getSeatsAvailable());
```

```
        System.out.println("Total Booked: " +
CinemaShow.getTotalBooked());
    }
}
```

2.3) (For Advanced Learners – home) Parking lot (ParkingLot.java) (a) Create class ParkingLot with private int twoWheelers and fourWheelers, private final int twoCap and fourCap, and private static long revenue=0. (b) park(String type): for “two”, if twoWheelers < twoCap increment and add 20 to revenue; for “four”, if fourWheelers < fourCap increment and add 40; otherwise print “Full” and reject. (c) leave(String type): decrement the matching count but never below 0. (d) In main: simulate a sequence of park/leave events, printing each outcome, then print final occupancy and revenue. Expected output: parking into a full section is refused; revenue = 20×(two parked) + 40×(four parked).

```
public class ParkingLot {

    private int twoWheelers;
    private int fourWheelers;
    private final int twoCap;
    private final int fourCap;
    private static long revenue = 0;

    public ParkingLot(int twoCap, int fourCap) {
        this.twoCap = twoCap;
        this.fourCap = fourCap;
```

```
        this.twoWheelers = 0;
        this.fourWheelers = 0;
    }

    public void park(String type) {
        if (type.equals("two")) {
            if (twoWheelers < twoCap) {
                twoWheelers++;
                revenue += 20;
                System.out.println("Two-wheeler parked");
            } else {
                System.out.println("Full");
            }
        } else if (type.equals("four")) {
            if (fourWheelers < fourCap) {
                fourWheelers++;
                revenue += 40;
                System.out.println("Four-wheeler parked");
            } else {
                System.out.println("Full");
            }
        }
    }

    public void leave(String type) {
        if (type.equals("two")) {
```

```
            if (twoWheelers > 0) {
                twoWheelers--;
                System.out.println("Two-wheeler left");
            }
        } else if (type.equals("four")) {
            if (fourWheelers > 0) {
                fourWheelers--;
                System.out.println("Four-wheeler left");
            }
        }
    }

    public int getTwoWheelers() {
        return twoWheelers;
    }

    public int getFourWheelers() {
        return fourWheelers;
    }

    public static long getRevenue() {
        return revenue;
    }

    public static void main(String[] args) {
        ParkingLot p = new ParkingLot(2, 2);
```

```
        p.park("two");
        p.park("two");
        p.park("two");

        p.park("four");
        p.park("four");
        p.park("four");

        p.leave("two");
        p.leave("four");
        p.park("two");
        p.park("four");
       System.out.println("Two-Wheelers: " + p.getTwoWheelers());
        System.out.println("Four-Wheelers: " + p.getFourWheelers());
        System.out.println("Revenue: " + ParkingLot.getRevenue());
    }
}
```